Code Jam 2009 - Qualification Round

Analysis: Alien Language

First we store all the words in a 2 dimensional array.
After that, we read each pattern, parse it, and count how many words match.

One possible way of storing a pattern is a 2 dimensional array P[15][26]. P[i][j] is True only if the **i**-th token contains the **j**-th letter of the alphabet, otherwise False. In other words, P[i] is a bitmap of the letters contained by the **i**-th token.

Parsing can be done like this:
- read one character **c**
- if **c** is '(', read characters until we hit ')'. The characters read are the token.
else the token is the character **c**
- populate P[i] for the characters in the token

To count how many words match, we make sure that each letter **i** from the word is contained in the bitmap P[i].

Total complexity is O(N * L * D).

In some programming languages this can solved by transforming the pattern into a regular expression. For instance in python replace '(' and ')' with '[' and ']'.

Code Jam 2009 - Qualification Round

Analysis: Watersheds

For each cell, we need to determine its eventual sink. Then, to each group of cells that share the same sink, we need to assign a unique label.

The inputs to this problem are small enough for a simple brute-force simulation algorithm. Start with a cell and trace the path that water would take by applying the water flow rules repeatedly. Here is one possible solution in Python.

```
import sys

def ReadInts():
  return list(map(int, sys.stdin.readline().strip().split(" ")))

def Cross(a, b):
  for i in a:
    for j in b:
      yield (i, j)

def Neighbours(ui, uj, m, n):
  if ui - 1 >= 0: yield (ui - 1, uj)
  if uj - 1 >= 0: yield (ui, uj - 1)
  if uj + 1 < n: yield (ui, uj + 1)
  if ui + 1 < m: yield (ui + 1, uj)

N = ReadInts()[0]
for prob in xrange(1, N + 1):
  # Read the map
  (m, n) = ReadInts()
  maze = [ReadInts() for _ in xrange(m)]
  answer = [["" for _ in xrange(n)] for _ in xrange(m)]

  # The map from sinks to labels.
  label = {}
  next_label = 'a'

  # Brute force each cell.
  for (ui, uj) in Cross(xrange(m), xrange(n)):
    (i, j) = (-1, -1)
    (nexti, nextj) = (ui, uj)
    while (i, j) != (nexti, nextj):
      (i, j) = (nexti, nextj)
      for (vi, vj) in Neighbours(i, j, m, n):
        if maze[vi][vj] < maze[nexti][nextj]:
          (nexti, nextj) = (vi, vj)

    # Cell (ui, uj) drains to (i, j).
    if (i, j) not in label:
      label[(i, j)] = next_label
      next_label = chr(ord(next_label) + 1)
    answer[ui][uj] = label[(i, j)]

  # Output the labels.
```

```
print "Case #%d:" % prob
for i in xrange(m):
  print " ".join(answer[i])
```

Code Jam 2009 - Qualification Round

Analysis: Welcome to Code Jam

In this problem, every one is welcomed by a (somehow) standard dynamic programming problem.

The word we want to find is **S** = "welcome to code jam", in a long string **T**. In fact the solution is not very different when we want to find any **S**. It is actually illustrative to picture the cases for short words.

In case **S** is just a single character, you just need to count how many times this character appears in **T**. If **S** = "xy" is a string of length 2, instead of brute force all the possible positions, one can do it in linear time, start from left to the right. For each occurrence of 'y', one needs to know how many 'x's appeared before that 'y'.

The general solution follows this pattern. Let us again use **S** = "welcome to code jam" as an example. The formal solution will be clear from the example; and you can always download the good solutions (with nice programming techniques) from the scoreboard.

So, let us define, for each position i in **T**, $\mathbf{T}^{(i)}$ to be the string consists of the first i characters of **T**. And write

- Dp[i,1]: How many times we can find "w" in $\mathbf{T}^{(i)}$?
- Dp[i,2]: How many times we can find "we" in $\mathbf{T}^{(i)}$?
- Dp[i,3]: How many times we can find "wel" in $\mathbf{T}^{(i)}$?
- Dp[i,4]: How many times we can find "welc" in $\mathbf{T}^{(i)}$?
- ...
- Dp[i,18]: How many times we can find "welcome to code ja" in $\mathbf{T}^{(i)}$?
- Dp[i,19]: How many times we can find "welcome to code jam" in $\mathbf{T}^{(i)}$?

Assume Dp[i,j] is computed for each j, let us see how easy we can compute, say, Dp[i+1,4]:

1. If the (i+1)-th character of **T** is not 'c', then Dp[i+1,4] = Dp[i,4].
2. If the (i+1)-th character of **T** is 'c', then we can include all the "welc"s found in $\mathbf{T}^{(i)}$, as well as those "welc"s ends exactly on the (i+1)-th character, so Dp[i+1,4] = Dp[i,4] + Dp[i,3].

Finally, let **n** be the length of the text **T**, Dp[n,19] will be our answer.

That's it. Welcome to Code Jam; and we hoped you enjoyed this round.